```
园林景观三维可视化设计平台 V1.0
[padding]
# apps/api/main.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
import mimetypes
import os
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
[padding]
try:
    from .domain.auth import authenticate, issue_token
    from .domain.csv_tools import parse_csv
    from .domain.exporter import export_all
    from .domain.errors import ApiError
    from .domain.reporting import build_markdown_report
    from .domain.router import Router, build_request, handle_api_error,
json_response, text_response
    from .domain.static_files import build_manifest, build_static_response
    from .domain.store import ProjectStore
    from .domain.version_diff import diff_snapshots
except ImportError:  # run as script: `python apps/api/main.py`
    import sys
[padding]
    PROJECT_ROOT_FALLBACK = Path(__file__).resolve().parents[2]
    sys.path.insert(0, str(PROJECT_ROOT_FALLBACK))
    from apps.api.domain.auth import authenticate, issue_token
    from apps.api.domain.csv_tools import parse_csv
    from apps.api.domain.exporter import export_all
    from apps.api.domain.errors import ApiError
    from apps.api.domain.reporting import build_markdown_report
    from apps.api.domain.router import Router, build_request,
handle_api_error, json_response, text_response
    from apps.api.domain.static_files import build_manifest,
build_static_response
    from apps.api.domain.store import ProjectStore
    from apps.api.domain.version_diff import diff_snapshots
[padding]
[padding]
PROJECT_ROOT = Path(__file__).resolve().parents[2]
WEB_ROOT = PROJECT_ROOT / "apps" / "web"
STORE = ProjectStore(PROJECT_ROOT)
ROUTER = Router()
[padding]
[padding]
def _read_body(handler: BaseHTTPRequestHandler) -> bytes:
    length = int(handler.headers.get("Content-Length") or "0")
    if length <= 0:
        return b""
```

```
    return handler.rfile.read(length)
[padding]
[padding]
def _safe_join(root: Path, rel: str) -> Path | None:
    rel = rel.lstrip("/").replace("/", os.sep)
    target = (root / rel).resolve()
    try:
        target.relative_to(root.resolve())
    except ValueError:
        return None
    return target
[padding]
[padding]
class Handler(BaseHTTPRequestHandler):
    server_version = "Garden3D/1.0"
[padding]
    def log_message(self, fmt: str, *args) -> None:
        return
[padding]
    def do_GET(self) -> None:
        if self.path.startswith("/api/"):
            return self._handle_api()
[padding]
        resp = build_static_response(root=WEB_ROOT, request_path=self.path,
fallback="index.html")
        self._send(resp.status, resp.headers, resp.body)
[padding]
    def do_POST(self) -> None:
        if self.path.startswith("/api/"):
            return self._handle_api()
        self.send_response(405)
        self.end_headers()
[padding]
    def do_PUT(self) -> None:
        if self.path.startswith("/api/"):
            return self._handle_api()
        self.send_response(405)
        self.end_headers()
[padding]
    def _send(self, status: int, headers: dict[str, str], body: bytes) ->
None:
        self.send_response(status)
        for k, v in headers.items():
            self.send_header(k, v)
        self.end_headers()
        if body:
            self.wfile.write(body)
[padding]
    def _handle_api(self) -> None:
        try:
```

```
            req = build_request(self.command, self.path, {k: str(v) for k, v
in self.headers.items()}, _read_body(self))
            handler, params = ROUTER.match(req)
            status, headers, body = handler(req, params)
            self._send(status, headers, body)
        except ApiError as exc:
            status, headers, body = handle_api_error(exc)
            self._send(status, headers, body)
        except Exception as exc:
            status, headers, body = json_response(500, {"ok": False,
"error": "internal_error", "message": str(exc)})
            self._send(status, headers, body)
[padding]
[padding]
def _require_demo_role(req, allowed: set[str]) -> None:
    auth = (req.headers.get("Authorization") or "").strip()
    if not auth.startswith("Bearer demo-"):
        # Demo mode: allow missing auth for reads.
        return
    parts = auth.replace("Bearer ", "", 1).split("-")
    role = parts[1] if len(parts) >= 2 else ""
    if role and role not in allowed:
        raise ApiError(403, "forbidden", "forbidden")
[padding]
[padding]
def _api_health(req, params):
    return json_response(200, {"ok": True})
[padding]
[padding]
def _api_login(req, params):
    ctype = (req.headers.get("Content-Type") or "").lower()
    username = ""
    password = ""
    if "application/json" in ctype:
        payload = req.json()
        username = str(payload.get("username") or "").strip()
        password = str(payload.get("password") or "").strip()
    else:
        form = req.form()
        username = (form.get("username") or "").strip()
        password = (form.get("password") or "").strip()
    user = authenticate(username, password)
    token = issue_token(user)
    return json_response(200, {"ok": True, "token": token, "role": user.role}
)
[padding]
[padding]
def _api_sites(req, params):
    return json_response(200, {"ok": True, "items": STORE.list_sites()})
[padding]
```

```
[padding]
def _api_plans(req, params):
    return json_response(200, {"ok": True, "items": STORE.list_plans()})
[padding]
[padding]
def _api_versions(req, params):
    return json_response(200, {"ok": True, "items": STORE.list_versions()})
[padding]
[padding]
def _api_plants(req, params):
    return json_response(200, {"ok": True, "items": STORE.list_plants()})
[padding]
def _api_create_plant(req, params):
    _require_demo_role(req, {"admin", "designer"})
    item = STORE.create_plant(req.json())
    return json_response(201, {"ok": True, "item": item})
[padding]
[padding]
def _api_update_plant(req, params):
    _require_demo_role(req, {"admin", "designer"})
    plant_id = params["plantId"]
    item = STORE.update_plant(plant_id, req.json())
    return json_response(200, {"ok": True, "item": item})
[padding]
[padding]
def _api_delete_plant(req, params):
    _require_demo_role(req, {"admin"})
    plant_id = params["plantId"]
    STORE.delete_plant(plant_id)
    return json_response(200, {"ok": True})
[padding]
[padding]
def _api_import_plants(req, params):
    _require_demo_role(req, {"admin", "designer"})
    payload = req.json()
    text = str(payload.get("csv") or "")
    rows = parse_csv(text)
    result = STORE.import_plants_csv(rows)
    return json_response(200, {"ok": True, "result": result})
[padding]
[padding]
def _api_materials(req, params):
    return json_response(200, {"ok": True, "items": STORE.list_materials()})
[padding]
def _api_create_material(req, params):
    _require_demo_role(req, {"admin", "designer"})
    item = STORE.create_material(req.json())
    return json_response(201, {"ok": True, "item": item})
[padding]
[padding]
```

```
def _api_update_material(req, params):
    _require_demo_role(req, {"admin", "designer"})
    item = STORE.update_material(params["materialId"], req.json())
    return json_response(200, {"ok": True, "item": item})
[padding]
[padding]
def _api_delete_material(req, params):
    _require_demo_role(req, {"admin"})
    STORE.delete_material(params["materialId"])
    return json_response(200, {"ok": True})
[padding]
[padding]
def _api_lights(req, params):
    return json_response(200, {"ok": True, "items": STORE.list_lights()})
[padding]
def _api_create_light(req, params):
    _require_demo_role(req, {"admin", "designer"})
    item = STORE.create_light(req.json())
    return json_response(201, {"ok": True, "item": item})
[padding]
[padding]
def _api_update_light(req, params):
    _require_demo_role(req, {"admin", "designer"})
    item = STORE.update_light(params["lightId"], req.json())
    return json_response(200, {"ok": True, "item": item})
[padding]
[padding]
def _api_delete_light(req, params):
    _require_demo_role(req, {"admin"})
    STORE.delete_light(params["lightId"])
    return json_response(200, {"ok": True})
[padding]
[padding]
def _api_issues(req, params):
    return json_response(200, {"ok": True, "items": STORE.list_issues()})
[padding]
[padding]
def _api_create_issue(req, params):
    _require_demo_role(req, {"admin", "designer", "reviewer"})
    version_id = params["versionId"]
    payload = req.json()
    issue = STORE.create_issue(version_id, payload)
    return json_response(201, {"ok": True, "item": issue})
[padding]
[padding]
def _api_update_issue(req, params):
    _require_demo_role(req, {"admin", "designer"})
    issue_id = params["issueId"]
    payload = req.json()
    issue = STORE.update_issue(issue_id, payload)
```

```
    return json_response(200, {"ok": True, "item": issue})
[padding]
[padding]
def _api_pavings(req, params):
    return json_response(200, {"ok": True, "items": STORE.list_pavings()})
[padding]
[padding]
def _api_create_paving(req, params):
    _require_demo_role(req, {"admin", "designer"})
    plan_id = params["planId"]
    payload = req.json()
    paving = STORE.create_paving(plan_id, payload)
    return json_response(201, {"ok": True, "item": paving})
[padding]
[padding]
def _api_export(req, params):
    bundle = export_all(STORE)
    kind = (req.query.get("kind") or ["all"])[0]
    if kind == "plants":
        return text_response(200, bundle.plants_csv, content_type="text/csv;
charset=utf-8")
    if kind == "materials":
        return text_response(200, bundle.materials_csv, content_type=
"text/csv; charset=utf-8")
    if kind == "lights":
        return text_response(200, bundle.lights_csv, content_type="text/csv;
charset=utf-8")
    if kind == "all":
        combined = (
            "# plants\n"
            + bundle.plants_csv
            + "\n# materials\n"
            + bundle.materials_csv
            + "\n# lights\n"
            + bundle.lights_csv
        )
        return text_response(200, combined, content_type="text/plain;
charset=utf-8")
    raise ApiError(400, "bad_request", "unknown export kind")
[padding]
def _api_diff(req, params):
    before_id = (req.query.get("before") or [""])[0].strip()
    after_id = (req.query.get("after") or [""])[0].strip()
    if not before_id or not after_id:
        raise ApiError(400, "bad_request", "before/after are required")
    before = STORE.get_version(before_id).get("snapshot") or {}
    after = STORE.get_version(after_id).get("snapshot") or {}
    if not isinstance(before, dict) or not isinstance(after, dict):
        raise ApiError(400, "bad_request", "snapshot must be object")
    return json_response(200, {"ok": True, "diff": diff_snapshots(before,
```

```
after)})
[padding]
[padding]
def _api_report(req, params):
    text = build_markdown_report(STORE._state)  # local-only
    return text_response(200, text, content_type="text/markdown; charset=utf-
8")
[padding]
def _api_web_manifest(req, params):
    return json_response(200, {"ok": True, "manifest":
build_manifest(WEB_ROOT)})
ROUTER.add("GET", "/api/health", _api_health)
ROUTER.add("POST", "/api/auth/login", _api_login)
ROUTER.add("GET", "/api/sites", _api_sites)
ROUTER.add("GET", "/api/plans", _api_plans)
ROUTER.add("GET", "/api/versions", _api_versions)
ROUTER.add("GET", "/api/plants", _api_plants)
ROUTER.add("POST", "/api/plants", _api_create_plant)
ROUTER.add("PUT", "/api/plants/{plantId}", _api_update_plant)
ROUTER.add("POST", "/api/plants/{plantId}/delete", _api_delete_plant)
ROUTER.add("POST", "/api/plants/import", _api_import_plants)
ROUTER.add("GET", "/api/materials", _api_materials)
ROUTER.add("POST", "/api/materials", _api_create_material)
ROUTER.add("PUT", "/api/materials/{materialId}", _api_update_material)
ROUTER.add("POST", "/api/materials/{materialId}/delete",
_api_delete_material)
ROUTER.add("GET", "/api/lights", _api_lights)
ROUTER.add("POST", "/api/lights", _api_create_light)
ROUTER.add("PUT", "/api/lights/{lightId}", _api_update_light)
ROUTER.add("POST", "/api/lights/{lightId}/delete", _api_delete_light)
ROUTER.add("GET", "/api/issues", _api_issues)
ROUTER.add("POST", "/api/versions/{versionId}/issues", _api_create_issue)
ROUTER.add("PUT", "/api/issues/{issueId}", _api_update_issue)
ROUTER.add("GET", "/api/pavings", _api_pavings)
ROUTER.add("POST", "/api/plans/{planId}/pavings", _api_create_paving)
ROUTER.add("GET", "/api/export", _api_export)
ROUTER.add("GET", "/api/versions/diff", _api_diff)
ROUTER.add("GET", "/api/report", _api_report)
ROUTER.add("GET", "/api/web/manifest", _api_web_manifest)
[padding]
[padding]
def main() -> None:
    port = int(os.environ.get("APP_PORT") or "8010")
    server = ThreadingHTTPServer(("127.0.0.1", port), Handler)
    server.serve_forever()
[padding]
[padding]
if __name__ == "__main__":
    main()
# apps/api/domain/router.py
```

```
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
import json
from dataclasses import dataclass
from typing import Any, Callable
from urllib.parse import parse_qs, urlparse
[padding]
from .errors import ApiError, BadRequest, NotFound
[padding]
[padding]
@dataclass(frozen=True)
class Request:
    method: str
    path: str
    query: dict[str, list[str]]
    headers: dict[str, str]
    body: bytes
[padding]
    def json(self) -> dict[str, Any]:
        if not self.body:
            return {}
        try:
            return json.loads(self.body.decode("utf-8"))
        except Exception as exc:
            raise BadRequest("invalid_json") from exc
[padding]
    def form(self) -> dict[str, str]:
        raw = parse_qs(self.body.decode("utf-8", errors="ignore"))
        return {k: (v[0] if v else "") for k, v in raw.items()}
[padding]
[padding]
Response = tuple[int, dict[str, str], bytes]
HandlerFn = Callable[[Request, dict[str, str]], Response]
[padding]
[padding]
def json_response(status: int, payload: dict[str, Any]) -> Response:
    data = json.dumps(payload, ensure_ascii=False).encode("utf-8")
    return status, {"Content-Type": "application/json; charset=utf-8",
"Content-Length": str(len(data))}, data
[padding]
[padding]
def text_response(status: int, text: str, *, content_type: str = "text/plain;
charset=utf-8") -> Response:
    data = text.encode("utf-8")
    return status, {"Content-Type": content_type, "Content-Length":
str(len(data))}, data
[padding]
[padding]
class Router:
```

```
    def __init__(self) -> None:
        self._routes: list[tuple[str, str, HandlerFn]] = []
[padding]
    def add(self, method: str, pattern: str, handler: HandlerFn) -> None:
        self._routes.append((method.upper(), pattern, handler))
[padding]
    def match(self, req: Request) -> tuple[HandlerFn, dict[str, str]]:
        for method, pattern, fn in self._routes:
            if method != req.method.upper():
                continue
            params = self._match_pattern(pattern, req.path)
            if params is not None:
                return fn, params
        raise NotFound()
[padding]
    @staticmethod
    def _match_pattern(pattern: str, path: str) -> dict[str, str] | None:
        # Very small path param matcher: "/api/plans/{planId}/versions"
        p_parts = [p for p in pattern.split("/") if p]
        a_parts = [p for p in path.split("/") if p]
        if len(p_parts) != len(a_parts):
            return None
        params: dict[str, str] = {}
        for p, a in zip(p_parts, a_parts, strict=True):
            if p.startswith("{") and p.endswith("}"):
                params[p[1:-1]] = a
            elif p != a:
                return None
        return params
[padding]
[padding]
def build_request(method: str, raw_path: str, headers: dict[str, str], body:
bytes) -> Request:
    parsed = urlparse(raw_path)
    return Request(method=method, path=parsed.path, query=
parse_qs(parsed.query), headers=headers, body=body)
[padding]
[padding]
def handle_api_error(exc: ApiError) -> Response:
    return json_response(exc.status, {"ok": False, "error": exc.code,
"message": exc.message})
[padding]
# apps/api/routers/__init__.py
[padding]
# apps/api/domain/auth.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
import secrets
from dataclasses import dataclass
```

```
[padding]
from .errors import Unauthorized
[padding]
[padding]
@dataclass(frozen=True)
class User:
    username: str
    role: str
[padding]
[padding]
_USERS = {
    "admin": {"password": "admin123", "role": "admin"},
    "designer": {"password": "design123", "role": "designer"},
    "reviewer": {"password": "review123", "role": "reviewer"},
}
[padding]
[padding]
def authenticate(username: str, password: str) -> User:
    record = _USERS.get(username)
    if not record or record["password"] != password:
        raise Unauthorized("invalid_credentials", code="invalid_credentials")
[padding]
    return User(username=username, role=record["role"])
[padding]
[padding]
def issue_token(user: User) -> str:
    # Local demo token. Server keeps no session; token only used for UI
hints.
    return f"demo-{user.role}-{secrets.token_hex(8)}"
[padding]
# apps/api/domain/csv_tools.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
import csv
import io
from dataclasses import dataclass
from typing import Iterable
[padding]
from .errors import BadRequest
[padding]
[padding]
def sniff_dialect(text: str) -> csv.Dialect:
    sample = text[:4096]
    try:
        return csv.Sniffer().sniff(sample)
    except Exception:
        class _Default(csv.Dialect):
            delimiter = ","
            quotechar = '"'
```

```
            doublequote = True
            skipinitialspace = False
            lineterminator = "\n"
            quoting = csv.QUOTE_MINIMAL
    [padding]
        return _Default()
    [padding]
    [padding]
    def parse_csv(text: str) -> list[list[str]]:
        if not isinstance(text, str) or not text.strip():
            raise BadRequest("csv text is empty.")
        dialect = sniff_dialect(text)
        reader = csv.reader(io.StringIO(text), dialect=dialect)
        rows = []
        for row in reader:
            if not row:
                continue
            if all(not str(cell).strip() for cell in row):
                continue
            rows.append([str(cell).strip() for cell in row])
        if not rows:
            raise BadRequest("csv contains no rows.")
        return rows
    [padding]
    [padding]
    def csv_to_text(rows: Iterable[Iterable[object]]) -> str:
        buf = io.StringIO()
        writer = csv.writer(buf, lineterminator="\n")
        for row in rows:
            writer.writerow(["" if v is None else str(v) for v in row])
        return buf.getvalue()
    [padding]
    [padding]
    @dataclass(frozen=True)
    class Table:
        header: list[str]
        rows: list[list[str]]
    [padding]
        def col_index(self, name: str) -> int:
            try:
                return self.header.index(name)
            except ValueError as exc:
                raise BadRequest(f"missing column: {name}") from exc
    [padding]
        def get(self, row: list[str], name: str) -> str:
            idx = self.col_index(name)
            return row[idx] if idx < len(row) else ""
    [padding]
    [padding]
    def as_table(rows: list[list[str]]) -> Table:
```

```
    header = rows[0]
    if len(header) < 2:
        raise BadRequest("csv header too short.")
    body = rows[1:]
    return Table(header=header, rows=body)
[padding]
# apps/api/domain/errors.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
[padding]
class ApiError(Exception):
    def __init__(self, status: int, code: str, message: str) -> None:
        super().__init__(message)
        self.status = status
        self.code = code
        self.message = message
[padding]
[padding]
class BadRequest(ApiError):
    def __init__(self, message: str, *, code: str = "bad_request") -> None:
        super().__init__(400, code, message)
[padding]
[padding]
class Unauthorized(ApiError):
    def __init__(self, message: str = "unauthorized", *, code: str =
"unauthorized") -> None:
        super().__init__(401, code, message)
[padding]
[padding]
class NotFound(ApiError):
    def __init__(self, message: str = "not_found", *, code: str =
"not_found") -> None:
        super().__init__(404, code, message)
[padding]
[padding]
class Conflict(ApiError):
    def __init__(self, message: str = "conflict", *, code: str = "conflict")
-> None:
        super().__init__(409, code, message)
[padding]
# apps/api/domain/exporter.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
from dataclasses import dataclass
from io import StringIO
[padding]
from .errors import BadRequest
from .store import ProjectStore
```

```
    [padding]
    [padding]
    def _csv_escape(value: object) -> str:
        s = "" if value is None else str(value)
        s = s.replace('"', '""')
        return f'\"{s}\"'
    [padding]
    [padding]
    def to_csv(rows: list[list[object]]) -> str:
        out = StringIO()
        for row in rows:
            out.write(",".join(_csv_escape(v) for v in row))
            out.write("\n")
        return out.getvalue()
    [padding]
    [padding]
    @dataclass(frozen=True)
    class ExportBundle:
        plants_csv: str
        materials_csv: str
        lights_csv: str
    [padding]
    [padding]
    def export_all(store: ProjectStore) -> ExportBundle:
        plants = store.list_plants()
        materials = store.list_materials()
        lights = store.list_lights()
        if not plants or not materials or not lights:
            raise BadRequest("export requires non-empty
    plants/materials/lights.")
    [padding]
        plants_rows: list[list[object]] = [["编号", "中文名", "类别",
    "常绿/落叶", "冠幅范围(m)", "季相要点"]]
        for p in plants:
            evergreen = "常绿" if bool(p.get("evergreen")) else "落叶"
            crown = f"{p.get('crownMinM')}-{p.get('crownMaxM')}"
            plants_rows.append([p.get("code"), p.get("cnName"), p.get("category")
    , evergreen, crown, p.get("seasonHint")])
    [padding]
        materials_rows: list[list[object]] = [["编号", "名称", "单位", "用途"]]
        for m in materials:
            materials_rows.append([m.get("code"), m.get("name"), m.get("unit"),
    m.get("usage")])
    [padding]
        lights_rows: list[list[object]] = [["编号", "色温", "功率(W)", "数量"]]
        for l in lights:
            lights_rows.append([l.get("code"), l.get("cct"), l.get("watt"),
    l.get("qty")])
    [padding]
        return ExportBundle(plants_csv=to_csv(plants_rows), materials_csv=
```

```
    to_csv(materials_rows), lights_csv=to_csv(lights_rows))
[padding]
# apps/api/domain/geometry.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
from dataclasses import dataclass
from math import hypot
[padding]
from .errors import BadRequest
[padding]
[padding]
@dataclass(frozen=True)
class Point:
    x: float
    y: float
[padding]
[padding]
def parse_point(obj: object, *, name: str = "point") -> Point:
    if not isinstance(obj, dict):
        raise BadRequest(f"{name} must be an object with x/y.")
    x = obj.get("x")
    y = obj.get("y")
    if not isinstance(x, (int, float)) or not isinstance(y, (int, float)):
        raise BadRequest(f"{name}.x and {name}.y must be numbers.")
    return Point(float(x), float(y))
[padding]
[padding]
def polyline_length(points: list[Point]) -> float:
    if len(points) < 2:
        return 0.0
    total = 0.0
    for a, b in zip(points, points[1:], strict=False):
        total += hypot(b.x - a.x, b.y - a.y)
    return total
[padding]
[padding]
def polygon_area(points: list[Point]) -> float:
    if len(points) < 3:
        return 0.0
    s = 0.0
    for a, b in zip(points, points[1:] + [points[0]], strict=False):
        s += a.x * b.y - b.x * a.y
    return abs(s) * 0.5
[padding]
[padding]
def validate_polygon(points: list[Point]) -> None:
    if len(points) < 3:
        raise BadRequest("polygon requires >= 3 points.")
    if polygon_area(points) <= 1e-9:
```

```
        raise BadRequest("polygon area too small or degenerate.")
[padding]
# apps/api/domain/ids.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
import secrets
import time
[padding]
[padding]
def new_id(prefix: str) -> str:
    # Stable-enough for local use; easy to read in exported CSV.
    ts = int(time.time() * 1000)
    rnd = secrets.token_hex(3)
    return f"{prefix}-{ts:x}-{rnd}"
[padding]
# apps/api/domain/reporting.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
"""
Reporting utilities for local project review.
[padding]
This module focuses on deterministic, explainable summaries that can be used
in:
- internal QA before publishing a plan version;
- exporting a "review pack" for stakeholders;
- consistency checks between exported lists and on-screen data.
[padding]
The functions are intentionally written without external dependencies to
keep the
runtime self-contained.
"""
[padding]
from dataclasses import dataclass
from datetime import datetime
from typing import Any, Iterable
[padding]
from .errors import BadRequest
[padding]
[padding]
def _now() -> str:
    return datetime.now().replace(microsecond=0).isoformat(sep=" ")
[padding]
[padding]
def _as_list(value: object) -> list:
    if value is None:
        return []
    if isinstance(value, list):
        return value
```

```
        raise BadRequest("expected list")
    [padding]
    [padding]
    def _as_dict(value: object) -> dict:
        if value is None:
            return {}
        if isinstance(value, dict):
            return value
        raise BadRequest("expected object")
    [padding]
    [padding]
    def _norm_text(s: object) -> str:
        return ("" if s is None else str(s)).strip()
    [padding]
    [padding]
    def _safe_float(v: object, *, default: float = 0.0) -> float:
        if v is None:
            return default
        if isinstance(v, (int, float)):
            return float(v)
        try:
            return float(str(v).strip())
        except Exception:
            return default
    [padding]
    [padding]
    def _safe_int(v: object, *, default: int = 0) -> int:
        if v is None:
            return default
        if isinstance(v, bool):
            return int(v)
        if isinstance(v, (int, float)):
            return int(v)
        try:
            return int(float(str(v).strip()))
        except Exception:
            return default
    [padding]
    [padding]
    def _pct(part: int, total: int) -> str:
        if total <= 0:
            return "0%"
        return f"{round(part * 100.0 / total)}%"
    [padding]
    [padding]
    @dataclass(frozen=True)
    class Kpi:
        sites: int
        plans: int
        versions: int
```

```
    plants: int
    materials: int
    lights: int
    issues: int
    pavings: int
[padding]
    def to_dict(self) -> dict[str, int]:
        return {
            "sites": self.sites,
            "plans": self.plans,
            "versions": self.versions,
            "plants": self.plants,
            "materials": self.materials,
            "lights": self.lights,
            "issues": self.issues,
            "pavings": self.pavings,
        }
[padding]
[padding]
def compute_kpi(state: dict[str, Any]) -> Kpi:
    return Kpi(
        sites=len(_as_list(state.get("sites"))),
        plans=len(_as_list(state.get("plans"))),
        versions=len(_as_list(state.get("versions"))),
        plants=len(_as_list(state.get("plants"))),
        materials=len(_as_list(state.get("materials"))),
        lights=len(_as_list(state.get("lights"))),
        issues=len(_as_list(state.get("issues"))),
        pavings=len(_as_list(state.get("pavings"))),
    )
[padding]
[padding]
def group_by(items: Iterable[dict[str, Any]], key: str) -> dict[str,
list[dict[str, Any]]]:
    out: dict[str, list[dict[str, Any]]] = {}
    for it in items:
        k = _norm_text(it.get(key))
        out.setdefault(k, []).append(it)
    return out
[padding]
[padding]
def summarize_plants(plants: list[dict[str, Any]]) -> dict[str, Any]:
    by_category = group_by(plants, "category")
    evergreen_count = 0
    for p in plants:
        if bool(p.get("evergreen")):
            evergreen_count += 1
    summary = {
        "total": len(plants),
        "evergreen": evergreen_count,
```

```
        "deciduous": len(plants) - evergreen_count,
        "evergreenRatio": _pct(evergreen_count, len(plants)),
        "byCategory": {k or "(未分类)": len(v) for k, v in
sorted(by_category.items(), key=lambda kv: kv[0])},
    }
    crown_ranges = []
    for p in plants:
        mn = _safe_float(p.get("crownMinM"))
        mx = _safe_float(p.get("crownMaxM"))
        if mn > 0 and mx > 0 and mx >= mn:
            crown_ranges.append((mn, mx))
    if crown_ranges:
        summary["crownMinMin"] = min(mn for mn, _ in crown_ranges)
        summary["crownMaxMax"] = max(mx for _, mx in crown_ranges)
    return summary
[padding]
[padding]
def summarize_materials(materials: list[dict[str, Any]]) -> dict[str, Any]:
    by_unit = group_by(materials, "unit")
    return {
        "total": len(materials),
        "byUnit": {k or "(未定义单位)": len(v) for k, v in
sorted(by_unit.items(), key=lambda kv: kv[0])},
    }
[padding]
[padding]
def summarize_lights(lights: list[dict[str, Any]]) -> dict[str, Any]:
    total_qty = sum(_safe_int(l.get("qty")) for l in lights)
    by_cct = group_by(lights, "cct")
    return {
        "total": len(lights),
        "totalQty": total_qty,
        "byCct": {k or "(未定义色温)": len(v) for k, v in
sorted(by_cct.items(), key=lambda kv: kv[0])},
    }
[padding]
[padding]
def summarize_issues(issues: list[dict[str, Any]]) -> dict[str, Any]:
    by_status = group_by(issues, "status")
    by_tag = group_by(issues, "tag")
    by_priority = group_by(issues, "priority")
    return {
        "total": len(issues),
        "byStatus": {k or "(未定义状态)": len(v) for k, v in
sorted(by_status.items(), key=lambda kv: kv[0])},
        "byTag": {k or "(未定义标签)": len(v) for k, v in
sorted(by_tag.items(), key=lambda kv: kv[0])},
        "byPriority": {k or "(未定义优先级)": len(v) for k, v in
sorted(by_priority.items(), key=lambda kv: kv[0])},
    }
```

```
[padding]
[padding]
@dataclass(frozen=True)
class ConsistencyIssue:
    level: str  # info/warn/error
    topic: str
    message: str
[padding]
    def to_dict(self) -> dict[str, str]:
        return {"level": self.level, "topic": self.topic, "message":
self.message}
[padding]
[padding]
def check_code_uniqueness(items: list[dict[str, Any]], *, topic: str) ->
list[ConsistencyIssue]:
    seen: set[str] = set()
    issues: list[ConsistencyIssue] = []
    for it in items:
        code = _norm_text(it.get("code")).upper()
        if not code:
            issues.append(ConsistencyIssue("error", topic, "missing code"))
            continue
        if code in seen:
            issues.append(ConsistencyIssue("error", topic, f"duplicate code:
{code}"))
            continue
        seen.add(code)
    return issues
[padding]
[padding]
def check_issue_flow(issues: list[dict[str, Any]]) -> list[ConsistencyIssue]
:
    allowed = {"新建", "处理中", "已解决", "已关闭"}
    out: list[ConsistencyIssue] = []
    for it in issues:
        status = _norm_text(it.get("status"))
        if status and status not in allowed:
            out.append(ConsistencyIssue("warn", "issues", f"unknown status:
{status}"))
    return out
[padding]
[padding]
def check_theme_keywords(plans: list[dict[str, Any]]) ->
list[ConsistencyIssue]:
    out: list[ConsistencyIssue] = []
    for p in plans:
        theme = _norm_text(p.get("theme"))
        if not theme:
            out.append(ConsistencyIssue("warn", "plans", "theme is empty"))
        elif len(theme) > 20:
```

```
            out.append(ConsistencyIssue("info", "plans", "theme text is long;
consider shorter label"))
    return out
[padding]
[padding]
def run_consistency_checks(state: dict[str, Any]) -> list[ConsistencyIssue]:
    plants = _as_list(state.get("plants"))
    materials = _as_list(state.get("materials"))
    lights = _as_list(state.get("lights"))
    issues = _as_list(state.get("issues"))
    plans = _as_list(state.get("plans"))
[padding]
    results: list[ConsistencyIssue] = []
    results.extend(check_code_uniqueness(plants, topic="plants"))
    results.extend(check_code_uniqueness(materials, topic="materials"))
    results.extend(check_code_uniqueness(lights, topic="lights"))
    results.extend(check_issue_flow(issues))
    results.extend(check_theme_keywords(plans))
    return results
[padding]
[padding]
def build_markdown_report(state: dict[str, Any]) -> str:
    kpi = compute_kpi(state)
    plants = _as_list(state.get("plants"))
    materials = _as_list(state.get("materials"))
    lights = _as_list(state.get("lights"))
    issues = _as_list(state.get("issues"))
    checks = run_consistency_checks(state)
[padding]
    p_sum = summarize_plants(plants)
    m_sum = summarize_materials(materials)
    l_sum = summarize_lights(lights)
    i_sum = summarize_issues(issues)
[padding]
    lines: list[str] = []
    lines.append(f"# 项目自检报告")
    lines.append("")
    lines.append(f"- 生成时间：{_now()}")
    lines.append("")
    lines.append("## 1 关键指标")
    lines.append("")
    for k, v in kpi.to_dict().items():
        lines.append(f"- {k}: {v}")
    lines.append("")
    lines.append("## 2 植物库概览")
    lines.append("")
    lines.append(f"- 总数：{p_sum['total']}")
    lines.append(f"- 常绿/落叶：{p_sum['evergreen']}/{p_sum['deciduous']}
（常绿占比 {p_sum['evergreenRatio']}）")
    if "crownMinMin" in p_sum and "crownMaxMax" in p_sum:
```

```
        lines.append(f"- 冠幅范围覆盖：{p_sum['crownMinMin']}m ~
{p_sum['crownMaxMax']}m")
    lines.append("- 类别分布：")
    for k, v in p_sum["byCategory"].items():
        lines.append(f"  - {k}: {v}")
    lines.append("")
    lines.append("## 3 材料字典概览")
    lines.append("")
    lines.append(f"- 总数：{m_sum['total']}")
    lines.append("- 单位分布：")
    for k, v in m_sum["byUnit"].items():
        lines.append(f"  - {k}: {v}")
    lines.append("")
    lines.append("## 4 灯具清单概览")
    lines.append("")
    lines.append(f"- 条目数：{l_sum['total']}")
    lines.append(f"- 数量合计：{l_sum['totalQty']}")
    lines.append("- 色温分布：")
    for k, v in l_sum["byCct"].items():
        lines.append(f"  - {k}: {v}")
    lines.append("")
    lines.append("## 5 评审事项概览")
    lines.append("")
    lines.append(f"- 总数：{i_sum['total']}")
    lines.append("- 状态分布：")
    for k, v in i_sum["byStatus"].items():
        lines.append(f"  - {k}: {v}")
    lines.append("- 标签分布：")
    for k, v in i_sum["byTag"].items():
        lines.append(f"  - {k}: {v}")
    lines.append("- 优先级分布：")
    for k, v in i_sum["byPriority"].items():
        lines.append(f"  - {k}: {v}")
    lines.append("")
    lines.append("## 6 一致性检查")
    lines.append("")
    if not checks:
        lines.append("- 未发现问题。")
    else:
        for it in checks:
            lines.append(f"- [{it.level}] {it.topic}: {it.message}")
    lines.append("")
    lines.append("## 7 建议")
    lines.append("")
    lines.append("- 发布版本前，优先关闭高优先级事项，确保变更摘要覆盖关键对
象。")
    lines.append("- 清单导出字段应与施工深化口径一致，必要时补充规格/工艺/等
级字段。")
    lines.append("- 对于涉及安全与无障碍的调整，建议在评审事项中明确原因与验
收标准。")
```

```
    lines.append("")
    return "\n".join(lines) + "\n"
[padding]
# apps/api/domain/snapshot.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
"""
Snapshot utilities.
[padding]
Rationale:
- A plan "version" should be reviewable after data changes.
- A snapshot is a plain JSON object that contains the relevant collections.
- This module provides deterministic snapshot generation and validation
helpers.
"""
[padding]
from dataclasses import dataclass
from typing import Any
[padding]
from .errors import BadRequest
[padding]
[padding]
@dataclass(frozen=True)
class Snapshot:
    season: str
    plants: list[dict[str, Any]]
    materials: list[dict[str, Any]]
    lights: list[dict[str, Any]]
    issues: list[dict[str, Any]]
    pavings: list[dict[str, Any]]
[padding]
    def to_dict(self) -> dict[str, Any]:
        return {
            "season": self.season,
            "plants": list(self.plants),
            "materials": list(self.materials),
            "lights": list(self.lights),
            "issues": list(self.issues),
            "pavings": list(self.pavings),
        }
[padding]
[padding]
def _as_list(value: object, *, name: str) -> list:
    if value is None:
        return []
    if isinstance(value, list):
        return value
    raise BadRequest(f"{name} must be list")
[padding]
```

```
[padding]
def _as_str(value: object, *, name: str, default: str = "") -> str:
    if value is None:
        return default
    s = str(value).strip()
    return s
[padding]
[padding]
def build_snapshot(
    *,
    season: str,
    plants: list[dict[str, Any]],
    materials: list[dict[str, Any]],
    lights: list[dict[str, Any]],
    issues: list[dict[str, Any]],
    pavings: list[dict[str, Any]],
) -> Snapshot:
    season = _as_str(season, name="season", default="春") or "春"
    return Snapshot(
        season=season,
        plants=list(plants),
        materials=list(materials),
        lights=list(lights),
        issues=list(issues),
        pavings=list(pavings),
    )
[padding]
[padding]
def validate_snapshot(payload: dict[str, Any]) -> None:
    if not isinstance(payload, dict):
        raise BadRequest("snapshot must be object")
    season = _as_str(payload.get("season"), name="season", default="")
    if season and season not in {"春", "夏", "秋", "冬"}:
        raise BadRequest("season must be one of 春/夏/秋/冬")
[padding]
    for key in ("plants", "materials", "lights", "issues", "pavings"):
        _as_list(payload.get(key), name=key)
[padding]
[padding]
def summarize_snapshot(payload: dict[str, Any]) -> dict[str, Any]:
    validate_snapshot(payload)
    season = _as_str(payload.get("season"), name="season", default="春") or
"春"
    plants = _as_list(payload.get("plants"), name="plants")
    materials = _as_list(payload.get("materials"), name="materials")
    lights = _as_list(payload.get("lights"), name="lights")
    issues = _as_list(payload.get("issues"), name="issues")
    pavings = _as_list(payload.get("pavings"), name="pavings")
    return {
        "season": season,
```

```
        "counts": {
            "plants": len(plants),
            "materials": len(materials),
            "lights": len(lights),
            "issues": len(issues),
            "pavings": len(pavings),
        },
        "checks": {
            "hasPlants": len(plants) > 0,
            "hasMaterials": len(materials) > 0,
            "hasLights": len(lights) > 0,
        },
    }
[padding]
# apps/api/domain/static_files.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
"""
Static file helper used by the built-in HTTP server.
[padding]
Goals:
- keep path traversal safe;
- provide consistent content-type;
- support simple cache headers to reduce reload overhead during review;
- avoid external dependencies.
"""
[padding]
import hashlib
import mimetypes
import os
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable
[padding]
from .errors import NotFound
[padding]
[padding]
TEXT_PREFIXES = ("text/",)
EXTRA_TEXT_TYPES = {
    ".js": "text/javascript",
    ".mjs": "text/javascript",
    ".css": "text/css",
    ".html": "text/html",
    ".json": "application/json",
    ".md": "text/markdown",
    ".svg": "image/svg+xml",
}
[padding]
[padding]
```

```
def safe_join(root: Path, rel: str) -> Path | None:
    rel = rel.lstrip("/").replace("/", os.sep)
    target = (root / rel).resolve()
    try:
        target.relative_to(root.resolve())
    except ValueError:
        return None
    return target
[padding]
[padding]
def guess_content_type(path: Path) -> str:
    ctype, _ = mimetypes.guess_type(str(path))
    if ctype:
        return ctype
    return EXTRA_TEXT_TYPES.get(path.suffix.lower(), "application/octet-
stream")
[padding]
[padding]
def is_text_type(content_type: str) -> bool:
    return any(content_type.startswith(p) for p in TEXT_PREFIXES) or
content_type in EXTRA_TEXT_TYPES.values()
[padding]
[padding]
def compute_etag(data: bytes) -> str:
    # Weak etag is fine for local use.
    digest = hashlib.sha1(data).hexdigest()
    return f'W/\"{digest}\"'
[padding]
[padding]
@dataclass(frozen=True)
class StaticResponse:
    status: int
    headers: dict[str, str]
    body: bytes
[padding]
[padding]
def build_static_response(
    *,
    root: Path,
    request_path: str,
    fallback: str = "index.html",
    cache_seconds: int = 30,
) -> StaticResponse:
    # Default "/" to index.
    path = request_path
    if path == "/" or path == "":
        path = "/" + fallback
    target = safe_join(root, path)
    if target is None or not target.exists() or not target.is_file():
        # SPA fallback
```

```
        target = root / fallback
        if not target.exists():
            raise NotFound("static file not found")
[padding]
    data = target.read_bytes()
    ctype = guess_content_type(target)
    headers = {
        "Content-Type": f"{ctype}; charset=utf-8" if is_text_type(ctype)
else ctype,
        "Content-Length": str(len(data)),
        "Cache-Control": f"public, max-age={max(cache_seconds,0)}",
        "ETag": compute_etag(data),
    }
    return StaticResponse(status=200, headers=headers, body=data)
[padding]
[padding]
def iter_files(root: Path, *, suffixes: Iterable[str] | None = None) ->
list[Path]:
    out: list[Path] = []
    if not root.exists():
        return out
    for p in sorted(root.rglob("*")):
        if not p.is_file():
            continue
        if suffixes is not None and p.suffix.lower() not in {s.lower() for s
in suffixes}:
            continue
        out.append(p)
    return out
[padding]
[padding]
def build_manifest(root: Path) -> dict[str, str]:
    """
    Return {relative_path: etag} for quick integrity checks.
    """
    manifest: dict[str, str] = {}
    for p in iter_files(root):
        rel = str(p.relative_to(root)).replace("\\", "/")
        try:
            manifest[rel] = compute_etag(p.read_bytes())
        except Exception:
            manifest[rel] = "W/\"unreadable\""
    return manifest
[padding]
# apps/api/domain/store.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
import json
from dataclasses import dataclass, field
```

```
from datetime import datetime
from pathlib import Path
from typing import Any
[padding]
from .errors import BadRequest, Conflict, NotFound
from .geometry import Point, parse_point, polygon_area, validate_polygon
from .ids import new_id
from .validators import LightInput, MaterialInput, PlantInput,
parse_light_input, parse_material_input, parse_plant_input, validate_code
[padding]
[padding]
def _now_iso() -> str:
    return datetime.now().replace(microsecond=0).isoformat(sep=" ")
[padding]
[padding]
def _read_json(path: Path, *, default: Any) -> Any:
    if not path.exists():
        return default
    return json.loads(path.read_text(encoding="utf-8"))
[padding]
[padding]
def _write_json(path: Path, payload: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
encoding="utf-8")
[padding]
[padding]
@dataclass
class Site:
    id: str
    name: str
    location: str
    scale_meter_per_px: float
    north_angle_deg: float
    created_at: str
[padding]
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "name": self.name,
            "location": self.location,
            "scaleMeterPerPx": self.scale_meter_per_px,
            "northAngleDeg": self.north_angle_deg,
            "createdAt": self.created_at,
        }
[padding]
[padding]
@dataclass
class Plan:
    id: str
```

```
    site_id: str
    name: str
    theme: str
    created_by: str
    created_at: str
[padding]
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "siteId": self.site_id,
            "name": self.name,
            "theme": self.theme,
            "createdBy": self.created_by,
            "createdAt": self.created_at,
        }
[padding]
[padding]
@dataclass
class PlanVersion:
    id: str
    plan_id: str
    version: str
    change_log: str
    created_at: str
    snapshot: dict[str, Any] = field(default_factory=dict)
[padding]
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "planId": self.plan_id,
            "version": self.version,
            "changeLog": self.change_log,
            "createdAt": self.created_at,
            "snapshot": self.snapshot,
        }
[padding]
[padding]
@dataclass
class PlantCatalogItem:
    id: str
    code: str
    cn_name: str
    category: str
    evergreen: bool
    crown_min_m: float
    crown_max_m: float
    season_hint: str
[padding]
    def to_dict(self) -> dict[str, Any]:
        return {
```

```
            "id": self.id,
            "code": self.code,
            "cnName": self.cn_name,
            "category": self.category,
            "evergreen": self.evergreen,
            "crownMinM": self.crown_min_m,
            "crownMaxM": self.crown_max_m,
            "seasonHint": self.season_hint,
        }
[padding]
[padding]
@dataclass
class MaterialItem:
    id: str
    code: str
    name: str
    unit: str
    usage: str
[padding]
    def to_dict(self) -> dict[str, Any]:
        return {"id": self.id, "code": self.code, "name": self.name, "unit":
self.unit, "usage": self.usage}
[padding]
[padding]
@dataclass
class LightItem:
    id: str
    code: str
    cct: str
    watt: int
    qty: int
[padding]
    def to_dict(self) -> dict[str, Any]:
        return {"id": self.id, "code": self.code, "cct": self.cct, "watt":
self.watt, "qty": self.qty}
[padding]
[padding]
@dataclass
class Issue:
    id: str
    version_id: str
    title: str
    tag: str
    priority: str
    status: str
    created_at: str
[padding]
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
```

```
            "versionId": self.version_id,
            "title": self.title,
            "tag": self.tag,
            "priority": self.priority,
            "status": self.status,
            "createdAt": self.created_at,
        }
[padding]
[padding]
@dataclass
class PavingSurface:
    id: str
    plan_id: str
    material_code: str
    points: list[Point]
[padding]
    def area_m2(self, *, meter_per_px: float) -> float:
        return polygon_area(self.points) * (meter_per_px**2)
[padding]
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "planId": self.plan_id,
            "materialCode": self.material_code,
            "points": [{"x": p.x, "y": p.y} for p in self.points],
        }
[padding]
[padding]
class ProjectStore:
    def __init__(self, base_dir: Path) -> None:
        self.base_dir = base_dir
        self.data_dir = base_dir / "apps" / "api" / "data"
        self.state_path = self.data_dir / "project_state.json"
        self._state = _read_json(self.state_path, default={})
        self._ensure_seed()
[padding]
    def _ensure_seed(self) -> None:
        if self._state.get("seeded"):
            return
        site_id = new_id("S")
        plan_id = new_id("PLN")
        version_id = new_id("V")
        self._state = {
            "seeded": True,
            "sites": [
                Site(
                    id=site_id,
                    name="滨水公园示范段",
                    location="华东 · 城市新区",
                    scale_meter_per_px=0.05,
```

```
            north_angle_deg=0.0,
    "latin": "Ophiopogon japonicus",
    "category": "地被",
    "evergreen": true,
    "crownMinM": 0.15,
    "crownMaxM": 0.25,
    "flowering": "",
    "seasonHint": "常绿耐阴，适合林下与边界收口。",
    "palette": { "春": "#4f7f55", "夏": "#2f6f4e", "秋": "#557a4a", "冬":
"#2a4a33" }
  },
[padding]
  {
    "code": "PL-0021",
    "cnName": "示例植物 021（滨水乔木）",
    "latin": "",
    "category": "乔木",
    "evergreen": false,
    "crownMinM": 4,
    "crownMaxM": 7,
    "flowering": "",
    "seasonHint": "用于滨水带背景林的示例条目。",
    "palette": { "春": "#6d8f58", "夏": "#4b7447", "秋": "#a38743", "冬":
"#6a5a52" }
  },
  {
    "code": "PL-0022",
    "cnName": "示例植物 022（观花小乔）",
    "latin": "",
    "category": "乔木",
    "evergreen": false,
    "crownMinM": 3,
    "crownMaxM": 5,
    "flowering": "3-5 月",
    "seasonHint": "用于花径与节点的示例条目。",
    "palette": { "春": "#e7b2c7", "夏": "#4d7a4b", "秋": "#8e6a56", "冬":
"#6b5a53" }
  },
  {
    "code": "PL-0023",
    "cnName": "示例植物 023（常绿灌木）",
    "latin": "",
    "category": "灌木",
    "evergreen": true,
    "crownMinM": 0.8,
    "crownMaxM": 1.6,
    "flowering": "",
    "seasonHint": "用于道路边界与基础绿篱的示例条目。",
    "palette": { "春": "#5f8a62", "夏": "#3e6f46", "秋": "#5c7a4e", "冬":
"#2f4a34" }
```

```
    },
    {
      "code": "PL-0024",
      "cnName": "示例植物 024（彩叶灌木）",
      "latin": "",
      "category": "灌木",
      "evergreen": true,
      "crownMinM": 0.6,
      "crownMaxM": 1.4,
      "flowering": "",
      "seasonHint": "用于色块与节点强调的示例条目。",
      "palette": { "春": "#7a3b56", "夏": "#6b2f4a", "秋": "#7a3b56", "冬":
"#4a2a3b" }
    },
    {
      "code": "PL-0025",
      "cnName": "示例植物 025（地被边界）",
      "latin": "",
      "category": "地被",
      "evergreen": true,
      "crownMinM": 0.12,
      "crownMaxM": 0.25,
      "flowering": "",
      "seasonHint": "用于铺装边界收口的示例条目。",
      "palette": { "春": "#4f7f55", "夏": "#2f6f4e", "秋": "#557a4a", "冬":
"#2a4a33" }
    }
  ]
}
# apps/api/data/project_state.json
{
  "seeded": true,
  "sites": [
    {
      "id": "S-19e6ea1e014-782e03",
      "name": "滨水公园示范段",
      "location": "华东 · 城市新区",
      "scaleMeterPerPx": 0.05,
      "northAngleDeg": 0.0,
      "createdAt": "2026-05-28 20:49:22"
    }
  ],
  "plans": [
    {
      "id": "PLN-19e6ea1e014-6b289d",
      "siteId": "S-19e6ea1e014-782e03",
      "name": "自然雅致方案",
      "theme": "自然雅致",
      "createdBy": "designer",
      "createdAt": "2026-05-28 20:49:22"
```

```
    }
   ],
   "versions": [
    {
     "id": "V-19e6ea1e014-973906",
     "planId": "PLN-19e6ea1e014-6b289d",
     "version": "V1.0.0",
     "changeLog": "初版：建立场地基准，整理植物与材料口径，形成评审事项清单
。",
     "createdAt": "2026-05-28 20:49:22",
     "snapshot": {
      "season": "春"
     }
    }
   ],
   "plants": [
    {
     "id": "P-19e6ea1e014-c0180c",
     "code": "P-001",
     "cnName": "香樟",
     "category": "乔木",
     "evergreen": true,
     "crownMinM": 6,
     "crownMaxM": 10,
     "seasonHint": "四季常绿，适合作为骨架乔木。"
    },
    {
     "id": "P-19e6ea1e014-e350e2",
     "code": "P-002",
     "cnName": "紫薇",
     "category": "乔木",
     "evergreen": false,
     "crownMinM": 3,
     "crownMaxM": 5,
     "seasonHint": "夏季花期明显，适合节点色彩强调。"
    },
    {
     "id": "P-19e6ea1e014-0fc535",
     "code": "P-003",
     "cnName": "红枫",
     "category": "乔木",
     "evergreen": false,
     "crownMinM": 4,
     "crownMaxM": 7,
     "seasonHint": "秋季红叶，宜与常绿灌木形成对比。"
    },
    {
     "id": "P-19e6ea1e014-7e5f98",
     "code": "P-004",
     "cnName": "海桐",
```

```
      "category": "灌木",
      "evergreen": true,
      "crownMinM": 0.8,
      "crownMaxM": 1.2,
      "seasonHint": "常绿耐修剪，可用于分隔与基础绿篱。"
    }
  ],
  "materials": [
    {
      "id": "M-19e6ea1e014-3f2c1f",
      "code": "M-01",
      "name": "透水砖（暖灰）",
      "unit": "㎡",
      "usage": "主园路"
    },
    {
      "id": "M-19e6ea1e014-3fc23b",
      "code": "M-02",
      "name": "花岗岩火烧面",
      "unit": "㎡",
      "usage": "入口广场"
    },
    {
      "id": "M-19e6ea1e014-594aa8",
      "code": "M-03",
      "name": "防腐木平台板",
      "unit": "㎡",
      "usage": "亲水平台"
    }
  ],
  "lights": [
    {
      "id": "L-19e6ea1e014-140b2f",
      "code": "L-01",
      "cct": "3000K",
      "watt": 12,
      "qty": 18
    },
    {
      "id": "L-19e6ea1e014-819d37",
      "code": "L-02",
      "cct": "3500K",
      "watt": 18,
      "qty": 6
    }
  ],
  "issues": [
    {
      "id": "R-1001",
      "versionId": "V-19e6ea1e014-973906",
```

```
      "title": "入口动线转角偏急",
      "tag": "动线",
      "priority": "高",
      "status": "新建",
      "createdAt": "2026-05-28 20:49:22"
    },
    {
      "id": "R-1002",
      "versionId": "V-19e6ea1e014-973906",
      "title": "水体边缘需增加防滑提示",
      "tag": "水体",
      "priority": "中",
      "status": "处理中",
      "createdAt": "2026-05-28 20:49:22"
    }
  ],
  "pavings": [
    {
      "id": "PV-19e6ea1e014-369dda",
      "planId": "PLN-19e6ea1e014-6b289d",
      "materialCode": "M-01",
      "points": [
        {
          "x": 10,
          "y": 10
        },
        {
          "x": 210,
          "y": 15
        },
        {
          "x": 230,
          "y": 120
        },
        {
          "x": 15,
          "y": 140
        }
      ]
    }
  ]
}
# apps/web/admin.html
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>系统管理 - 园林景观三维可视化设计平台</title>
    <meta http-equiv="refresh" content="0; url=./index.html#site" />
```

```
  </head>
  <body>
    <p>正在跳转到系统管理…</p>
  </body>
</html>
# apps/web/app.css
:root{
  --bg:#f6f7f4;
  --panel:#ffffff;
  --ink:#1f2a24;
  --muted:#55635b;
  --accent:#2f6f4e;
  --accent2:#9db9a7;
  --border:#e2e7e2;
  --shadow:0 10px 30px rgba(18,24,20,.08);
  --radius:14px;
}
*{box-sizing:border-box}
body{
  margin:0;
  font-family: "Microsoft YaHei", "PingFang SC", system-ui, -apple-system,
Segoe UI, Arial, sans-serif;
  color:var(--ink);
  background:linear-gradient(180deg,#eef2ee 0%, var(--bg) 40%, #f9faf8 100%);
[padding]
}
a{color:var(--accent); text-decoration:none}
header{
  position:sticky; top:0; z-index:10;
  background:rgba(246,247,244,.86);
  backdrop-filter: blur(10px);
  border-bottom:1px solid var(--border);
}
.topbar{
  max-width:1200px; margin:0 auto;
  padding:14px 18px;
  display:flex; gap:12px; align-items:center; justify-content:space-between;
}
.brand{
  display:flex; gap:10px; align-items:center;
}
.logo{
  width:34px; height:34px; border-radius:10px;
  background: radial-gradient(circle at 30% 30%, #b8d6c4 0%, #2f6f4e 55%,
#254a38 100%);
  box-shadow: 0 8px 20px rgba(47,111,78,.25);
}
.brand h1{
  font-size:16px; margin:0;
}
```

```
.brand .sub{
  font-size:12px; color:var(--muted);
}
.nav{
  display:flex; gap:10px; flex-wrap:wrap;
}
.nav a{
  padding:8px 10px;
  border-radius:10px;
  color:var(--muted);
}
.nav a.active{
  color:var(--ink);
  background:rgba(47,111,78,.10);
  border:1px solid rgba(47,111,78,.18);
}
.wrap{max-width:1200px; margin:0 auto; padding:18px}
.grid{
  display:grid;
  grid-template-columns: 1fr;
  gap:14px;
}
@media (min-width: 980px){
  .grid{grid-template-columns: 340px 1fr}
}
.card{
  background:var(--panel);
  border:1px solid var(--border);
  border-radius:var(--radius);
  box-shadow:var(--shadow);
}
.card .hd{
  padding:14px 16px;
  border-bottom:1px solid var(--border);
  display:flex; align-items:center; justify-content:space-between; gap:10px;
}
.card .hd h2{font-size:14px; margin:0}
.card .bd{padding:14px 16px}
.kpi{
  display:grid; grid-template-columns:repeat(2,1fr); gap:10px;
}
.kpi .box{
  border:1px solid var(--border);
  border-radius:12px;
  padding:10px;
  background:linear-gradient(180deg,#fff 0%, #fbfcfb 100%);
}
.kpi .num{font-size:18px; font-weight:700}
.kpi .lab{font-size:12px; color:var(--muted)}
.btn{
```

```
  appearance:none;
  border:1px solid rgba(47,111,78,.25);
  background:rgba(47,111,78,.10);
  color:var(--ink);
  border-radius:12px;
  padding:9px 12px;
  cursor:pointer;
}
.btn.primary{
  background:linear-gradient(180deg,#3a815b 0%, #2f6f4e 100%);
  color:#fff;
  border-color:rgba(0,0,0,0);
}
.btn.danger{
  background:rgba(159,45,45,.08);
  border-color:rgba(159,45,45,.22);
}
.row{display:flex; gap:10px; align-items:center; flex-wrap:wrap}
label{font-size:12px; color:var(--muted)}
input,select,textarea{
  width:100%;
  padding:10px 12px;
  border:1px solid var(--border);
  border-radius:12px;
  outline:none;
  background:#fff;
}
textarea{min-height:90px; resize:vertical}
.field{display:grid; gap:6px; margin-bottom:10px}
.muted{color:var(--muted); font-size:12px}
table{width:100%; border-collapse:collapse}
th,td{padding:10px 8px; border-bottom:1px solid var(--border); font-
size:13px; text-align:left}
th{color:var(--muted); font-weight:600; background:#fbfcfb}
.pill{
  display:inline-flex; align-items:center; gap:6px;
  padding:4px 10px; border-radius:999px;
  border:1px solid rgba(47,111,78,.20);
  background:rgba(47,111,78,.08);
  font-size:12px;
}
.view-title{
  display:flex; align-items:flex-start; justify-content:space-between;
gap:12px;
}
.view-title h3{margin:0; font-size:16px}
.view-title p{margin:4px 0 0; font-size:12px; color:var(--muted)}
[padding]
.modal-shell{
  position:fixed; inset:0; display:none;
```

```
  background:rgba(10,12,11,.32);
  z-index:50;
 }
 .modal-shell.modal{display:flex; align-items:center; justify-content:center;
 padding:18px}
 .modal{
  width:min(760px, 96vw);
  background:#fff;
  border-radius:18px;
  border:1px solid var(--border);
  box-shadow: 0 18px 40px rgba(0,0,0,.18);
  overflow:hidden;
 }
 .modal .mhd{
  padding:14px 16px;
  border-bottom:1px solid var(--border);
  display:flex; align-items:center; justify-content:space-between; gap:10px;
 }
 .modal .mhd strong{font-size:14px}
 .modal .mbd{padding:14px 16px}
 .modal .mft{
  padding:14px 16px;
  border-top:1px solid var(--border);
  display:flex; justify-content:flex-end; gap:10px;
 }
 [padding]
 .split{
  display:grid; grid-template-columns:1fr; gap:12px;
 }
 @media (min-width: 820px){
  .split{grid-template-columns: 1fr 1fr}
 }
 [padding]
 # apps/web/app.js
 const App = (() => {
  const state = {
   token: localStorage.getItem("g3d_token") || "",
   role: localStorage.getItem("g3d_role") || "",
   season: "春",
   plants: [],
   materials: [],
   lights: [],
   issues: [],
   versions: [],
  };
 [padding]
  async function apiGet(path) {
   const headers = {};
   if (state.token) headers["Authorization"] = `Bearer ${state.token}`;
   const res = await fetch(path, { headers });
```

```
    const data = await res.json().catch(() => ({}));
    if (!res.ok || data.ok === false) throw new Error(data.error ||
"api_failed");
    return data;
  }
[padding]
  async function loadCoreData() {
    const [plants, materials, lights, issues, versions] = await
Promise.all([
      apiGet("/api/plants"),
      apiGet("/api/materials"),
      apiGet("/api/lights"),
      apiGet("/api/issues"),
      apiGet("/api/versions"),
    ]);
    state.plants = plants.items || [];
    state.materials = materials.items || [];
    state.lights = lights.items || [];
    state.issues = issues.items || [];
    state.versions = versions.items || [];
  }
[padding]
  const views = {
    home: {
      title: "方案总览",
      desc: "聚焦场地、版本、评审与清单输出。",
      render: () => `
        <div class="grid">
          <section class="card">
            <div class="hd"><h2>快速入口</h2><span class=
"pill">自然雅致主题</span></div>
            <div class="bd">
              <div class="row">
                <a class="btn primary" href="#site">进入场地</a>
                <a class="btn" href="#plan">方案编辑</a>
                <a class="btn" href="#review">评审中心</a>
                <a class="btn" href="#export">清单导出</a>
              </div>
              <div style="height:10px"></div>
              <div class="muted">演示账号：admin/admin123（用于自动截图与门
禁）。</div>
            </div>
          </section>
          <section class="card">
            <div class="hd"><h2>关键指标</h2><span class="muted">V1.0
数据示例</span></div>
            <div class="bd">
              <div class="kpi">
                <div class="box"><div class="num">1</div><div class=
"lab">场地</div></div>
```

```
            <div class="box"><div class="num">3</div><div class=
"lab">方案版本</div></div>
            <div class="box"><div class="num">${state.plants.length}
</div><div class="lab">植物品种</div></div>
            <div class="box"><div class="num">${state.issues.length}
</div><div class="lab">评审事项</div></div>
          </div>
        </div>
      </section>
    </div>
  `,
 },
 site: {
  title: "场地与底图",
  desc: "底图标定、北向与比例尺，作为三维场景的基准。",
  render: () => `
   <section class="card">
    <div class="hd"><h2>场地信息</h2><span class=
"pill">基准：平面坐标</span></div>
    <div class="bd split">
     <div>
      <div class="field"><label>场地名称</label><input value=
"滨水公园示范段" readonly /></div>
      <div class="field"><label>位置</label><input value="华东 ·
城市新区" readonly /></div>
      <div class="field"><label>比例尺标定</label><input value="1px =
0.05m（两点测距标定）" readonly /></div>
      <div class="muted">提示：V1.0 以快速可视化与清单闭环为主，地形
与构件为轻量参数化对象。</div>
     </div>
     <div>
      <div class="field"><label>底图（示意）</label><div class=
"card" style="border-style:dashed; box-shadow:none; padding:14px;">
      </div></div>
     </div>
    </div>
   </section>
  `,
 },
 plan: {
  title: "方案编辑",
  desc: "道路、铺装、水体、构筑物与季相预览。",
  render: () => `
   <section class="card">
    <div class="hd">
     <h2>编辑面板</h2>
     <div class="row">
      <span class="pill">季相：${state.season}</span>
      <button class="btn" id="season-btn">切换季节</button>
      <a class="btn" href="#planting">植物布置</a>
```

```
            <a class="btn" href="#materials">材质库</a>
          </div>
        </div>
        <div class="bd">
          <div class="view-title">
            <div>
              <h3>三维视窗（演示）</h3>
              <p>真实三维渲染在 V1.0 规划中采用浏览器
WebGL；此处用“场景要素清单”替代，以便截图、文档与代码闭环。</p>
            </div>
            <span class="pill">版本：V1.0.0</span>
          </div>
          <div style="height:8px"></div>
          <table>
            <thead><tr><th>要素</th><th>参数</th><th>说明</th></tr></thead
>
            <tbody>
              <tr><td>道路</td><td>中心线+宽度
3.5m</td><td>主园路，圆角过渡</td></tr>
              <tr><td>铺装面</td><td>透水砖（暖灰）</td><td>入口广场与节点
铺装</td></tr>
              <tr><td>水体</td><td>水面高程 +
0.2m</td><td>浅水景观与亲水平台</td></tr>
              <tr><td>灯光</td><td>3000K 暖光</td><td>夜景路径引导</td></t
r>
            </tbody>
          </table>
        </div>
      </section>
    `,
    bind: () => {
      const btn = document.getElementById("season-btn");
      if (!btn) return;
      btn.addEventListener("click", () => {
        const order = ["春", "夏", "秋", "冬"];
        const idx = order.indexOf(state.season);
        state.season = order[(idx + 1) % order.length];
        render();
      });
    },
  },
  planting: {
    title: "植物库与布置",
    desc: "品种、规格与季相信息统一到清单中。",
    render: () => `
      <section class="card">
        <div class="hd">
          <h2>植物库（可评审）</h2>
          <div class="row">
            <button class="btn primary" data-action="create">新增</button>
```

```
            <button class="btn" data-action="edit">编辑</button>
            <button class="btn" data-action="view">查看</button>
            <button class="btn danger" data-action="delete">删除</button>
          </div>
        </div>
        <div class="bd">
          <div style="height:10px"></div>
          <table>
            <thead><tr><th>编号</th><th>中文名</th><th>类别</th><th>常绿/
落叶</th><th>冠幅</th><th>季相要点</th></tr></thead>
            <tbody>
              ${
                state.plants
                  .map((p) => {
                    const evergreen = p.evergreen ? "常绿" : "落叶";
                    const crown = `${p.crownMinM}-${p.crownMaxM}m`;
                    return `<tr><td>${p.code}</td><td>${p.cnName}
</td><td>${p.category}</td><td>${evergreen}</td><td>${crown}
</td><td>${p.seasonHint || ""}</td></tr>`;
                  })
                  .join("")
              }
            </tbody>
          </table>
        </div>
      </section>
    `,
    bind: () => {
      document.querySelectorAll('[data-action]').forEach((el) => {
        el.addEventListener("click", () =>
openPlantModal(el.getAttribute("data-action")));
      });
    },
  },
  materials: {
    title: "材料与铺装",
    desc: "把视觉效果与工程清单映射到同一份材料字典。",
    render: () => `
      <section class="card">
        <div class="hd"><h2>材料字典</h2><span class="pill">低饱和 ·
暖灰</span></div>
        <div class="bd">
          <table>
            <thead><tr><th>编号</th><th>名称</th><th>计量单位</th><th>用途
</th></tr></thead>
            <tbody>
              ${state.materials.map(m => `<tr><td>${m.code}
</td><td>${m.name}</td><td>${m.unit}</td><td>${m.usage}</td></tr>`).join("")}
[padding]
            </tbody>
```

```
          </table>
        </div>
      </section>
    `,
  },
  review: {
    title: "评审中心",
    desc: "意见基于截图流转，版本可追溯。",
    render: () => `
      <section class="card">
        <div class="hd"><h2>评审事项</h2><span class="pill">版本：V1.0.0</span></div>
        <div class="bd">
          <table>
            <thead><tr><th>编号</th><th>标题</th><th>标签</th><th>优先级</th><th>状态</th></tr></thead>
            <tbody>
              ${state.issues.map(i => `<tr><td>${i.id}</td><td>${i.title}</td><td>${i.tag}</td><td>${i.priority}</td><td>${i.status}</td></tr>`).join("")}
            </tbody>
          </table>
          <div style="height:10px"></div>
        </div>
      </section>
    `,
  },
  export: {
    title: "清单导出",
    desc: "植物、材料与灯具清单以 CSV 输出，支撑交付与招采。",
    render: () => `
      <section class="card">
        <div class="hd"><h2>导出面板</h2><span class="muted">示例导出</span></div>
        <div class="bd">
          <div class="row">
            <button class="btn primary" id="export-plants">导出植物清单</button>
            <button class="btn" id="export-materials">导出材料清单</button>
            <button class="btn" id="export-lights">导出灯具清单</button>
          </div>
          <div style="height:12px"></div>
          <pre id="export-out" class="card" style="box-shadow:none; background:#fbfcfb; border-style:dashed; padding:12px; white-space:pre-wrap; margin:0;"></pre>
        </div>
      </section>
    `,
    bind: () => {
```

```
        const out = document.getElementById("export-out");
        const write = (txt) => { if(out) out.textContent = txt; };
        const csv = (rows) => rows.map(r => r.map(v => `"${String(v)
.replaceAll("","""")}"`).join(",")).join("\n");
        document.getElementById("export-plants")?.addEventListener("click",
() => {
          write(csv([["编号","中文名","类别","常绿/落叶","冠幅范围(m)"],
...state.plants.map(p=>[p.code,p.cnName,p.category,(p.evergreen?
"常绿":"落叶"),`${p.crownMinM}-${p.crownMaxM}`])]));
        });
        document.getElementById("export-materials")
?.addEventListener("click", () => {
          write(csv([["编号","名称","单位","用途"], ...state.materials.map(m=
>[m.code,m.name,m.unit,m.usage])]));
        });
        document.getElementById("export-lights")?.addEventListener("click",
() => {
          write(csv([["编号","色温","功率(W)","数量"], ...state.lights.map(l=
>[l.code,l.cct,l.watt,l.qty])]));
        });
      },
    },
  };
[padding]
  function setActiveNav(hash) {
    document.querySelectorAll(".nav a").forEach((a) => {
      const target = (a.getAttribute("href") || "").replace(/^#/, "");
      a.classList.toggle("active", target === hash);
    });
  }
[padding]
  function currentHash() {
    const h = (location.hash || "").replace(/^#/, "");
    return h || "home";
  }
[padding]
  function ensureAuth() {
    const view = currentHash();
    if (!state.token && view !== "login") {
      location.hash = "login";
      return false;
    }
    return true;
  }
[padding]
  function openModal({ title, bodyHtml, footerButtons }) {
    const shell = document.getElementById("modal");
    shell.classList.add("modal");
    shell.innerHTML = `
      <div class="modal" role="dialog" aria-label="${escapeHtml(title)}">
```

```
        <div class="mhd">
          <strong>${escapeHtml(title)}</strong>
          <button class="btn" id="modal-close">关闭</button>
        </div>
        <div class="mbd">${bodyHtml}</div>
        <div class="mft">${footerButtons || `<button class="btn primary" id=
"modal-ok">确定</button>`}</div>
      </div>
    `;
    shell.querySelector("#modal-close")?.addEventListener("click",
closeModal);
    shell.querySelector("#modal-ok")?.addEventListener("click", closeModal);
  }
[padding]
  function closeModal() {
    const shell = document.getElementById("modal");
    shell.classList.remove("modal");
    shell.innerHTML = "";
  }
[padding]
  function openPlantModal(action) {
    const map = { create: "新增植物品种", edit: "编辑植物信息", view:
"查看植物详情", delete: "删除植物确认" };
    const title = map[action] || "植物操作";
    if (action === "delete") {
      return openModal({
        title,
        footerButtons: `<button class="btn" id="modal-
ok">取消</button><button class="btn danger" id="modal-
del">确认删除</button>`,
      });
    }
    const readonly = action === "view" ? "readonly" : "";
    const hint = action === "view" ? "（只读预览）" : "（示例表单）";
    openModal({
      title: `${title}${hint}`,
      bodyHtml: `
        <div class="split">
          <div class="field"><label>中文名</label><input value="紫薇"
${readonly} /></div>
          <div class="field"><label>类别</label><select ${readonly ?
"disabled" : ""}><option>乔木</option><option>灌木</option><option>地被</opt
ion></select></div>
          <div class="field"><label>常绿/落叶</label><select ${readonly ?
"disabled" : ""}><option>落叶</option><option>常绿</option></select></div>
          <div class="field"><label>冠幅范围</label><input value="3-5m"
${readonly} /></div>
        </div>
        <div class="field"><label>季相要点</label><textarea ${readonly}
>夏季花期明显，适合节点色彩强调；秋季叶色偏黄，建议与常绿灌木搭配。</textare
```

```
a></div>
      `,
    });
  }
[padding]
  function escapeHtml(s) {
    return String(s).replaceAll("&", "&amp;").replaceAll("<", "&lt;")
.replaceAll(">", "&gt;").replaceAll("", "&quot;");
  }
[padding]
  async function apiLogin(username, password) {
    const res = await fetch("/api/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ username, password }),
    });
    const data = await res.json().catch(() => ({}));
    if (!res.ok || !data.ok) throw new Error(data.error || "login_failed");
    return data;
  }
[padding]
  function bindLogin() {
    const form = document.getElementById("login-form");
    if (!form) return;
    form.addEventListener("submit", async (e) => {
      e.preventDefault();
      const u = form.querySelector('input[name="username"]').value.trim();
      const p = form.querySelector('input[name="password"]').value.trim();
      const msg = document.getElementById("login-msg");
      try {
        const data = await apiLogin(u, p);
        state.token = data.token;
        state.role = data.role;
        localStorage.setItem("g3d_token", state.token);
        localStorage.setItem("g3d_role", state.role);
        location.hash = "home";
      } catch (err) {
        if (msg) msg.textContent = "账号或密码不正确（演示：admin/admin123）
。";
      }
    });
  }
[padding]
  function renderLogin() {
    return `
      <div class="wrap">
        <section class="card" style="max-width:860px; margin:26px auto;">
          <div class="hd">
            <h2>登录</h2>
            <span class="pill">园林景观三维可视化设计平台</span>
```

```
        </div>
        <div class="bd">
          <div class="split">
            <div>
              <form id="login-form">
                <div class="field"><label>用户名</label><input name=
"username" autocomplete="username" value="admin" /></div>
                <div class="field"><label>密码</label><input name=
"password" type="password" autocomplete="current-password" value="admin123"
/></div>
                <div class="row">
                  <button class="btn primary" type="submit">登录</button>
                  <span id="login-msg" class="muted"></span>
                </div>
              </form>
            </div>
            <div>
              <div class="card" style="box-shadow:none; background:#fbfcfb;
border-style:dashed;">
                <div class="bd">
                  <div class="view-title">
                    <div>
                      <h3>演示说明</h3>
                    </div>
                    <span class="pill">V1.0</span>
                  </div>
                  <ul class="muted">
                    <li>设计师：designer/design123</li>
                    <li>评审人：reviewer/review123</li>
                    <li>管理员：admin/admin123</li>
                  </ul>
                  <div class="muted">视觉风格：低饱和绿 + 暖灰 +
克制阴影，突出绿植与场景留白。</div>
                </div>
              </div>
            </div>
          </div>
        </div>
      </section>
    </div>
  `;
 }
[padding]
 function renderShell(innerHtml) {
   const h = currentHash();
   const navItems = [
     ["home", "总览"],
     ["site", "场地"],
     ["plan", "方案"],
     ["planting", "植物"],
```

```
      ["materials", "材料"],
      ["review", "评审"],
      ["export", "导出"],
    ];
    return `
      <header>
        <div class="topbar">
          <div class="brand">
            <div class="logo" aria-hidden="true"></div>
            <div>
              <h1>园林景观三维可视化设计平台</h1>
              <div class="sub">自然雅致 · 版本 V1.0 · 登录角色：${state.role
|| "未登录"}</div>
            </div>
          </div>
          <nav class="nav" aria-label="主导航">
            ${navItems.map(([k, t]) => `<a href="#${k}" class="${k === h ?
"active" : ""}">${t}</a>`).join("")}
            <a href="#login" id="logout-link">退出</a>
          </nav>
        </div>
      </header>
      <div class="wrap">${innerHtml}</div>
      <div id="modal" class="modal-shell"></div>
    `;
  }
[padding]
  function bindShell() {
    document.getElementById("logout-link")?.addEventListener("click", () =>
{
      state.token = "";
      state.role = "";
      localStorage.removeItem("g3d_token");
      localStorage.removeItem("g3d_role");
      closeModal();
    });
  }
[padding]
  async function render() {
    const mount = document.getElementById("app");
    const h = currentHash();
    if (!mount) return;
[padding]
    if (h === "login") {
      mount.innerHTML = renderLogin();
      bindLogin();
      return;
    }
[padding]
    if (!ensureAuth()) return;
```

```
    if (!state.plants.length || !state.materials.length ||
!state.issues.length) {
      try {
        await loadCoreData();
      } catch (e) {
        // During static preview, api may be unavailable; keep UI usable.
      }
    }
    const view = views[h] || views.home;
    const inner = view.render();
    mount.innerHTML = renderShell(inner);
    setActiveNav(h);
    bindShell();
    view.bind?.();
  }
[padding]
  window.__rkModal = { closeModal };
  window.addEventListener("hashchange", render);
[padding]
  return { render };
})();
[padding]
window.addEventListener("DOMContentLoaded", () => App.render());
# apps/web/devtools.html
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>开发自检 - 园林景观三维可视化设计平台</title>
    <link rel="stylesheet" href="./app.css" />
  </head>
  <body>
    <header>
      <div class="topbar">
        <div class="brand">
          <div class="logo" aria-hidden="true"></div>
          <div>
            <h1>开发自检</h1>
            <div class="sub">用于本地接口与静态资源快速核对</div>
          </div>
        </div>
        <nav class="nav" aria-label="辅助导航">
          <a href="./index.html#home">返回系统</a>
        </nav>
      </div>
    </header>
    <div class="wrap">
      <section class="card">
        <div class="hd">
```

```
        <h2>接口自检</h2>
        <span class="muted">不写入数据，仅拉取查看</span>
      </div>
      <div class="bd">
        <div class="row">
          <button class="btn primary" id="btn-health">/api/health</button>
          <button class="btn" id="btn-report">/api/report</button>
          <button class="btn" id="btn-manifest">/api/web/manifest</button>
        </div>
        <div style="height:12px"></div>
        <pre id="out" class="card" style="box-shadow:none;
background:#fbfcfb; border-style:dashed; padding:12px; white-space:pre-wrap;
margin:0;"></pre>
      </div>
    </section>
[padding]
    <section class="card" style="margin-top:14px;">
      <div class="hd"><h2>说明</h2><span class=
"pill">本地运行</span></div>
      <div class="bd">
        <div class="muted">
          <p>该页用于检查：</p>
          <ul>
            <li>服务是否可用（health）</li>
            <li>项目汇总报告是否可生成（report）</li>
            <li>静态资源清单是否完整（manifest）</li>
          </ul>
          <p>若用于截图采集，仍以系统入口 <code>index.html</code>
为准。</p>
        </div>
      </div>
    </section>
  </div>
  <script src="./devtools.js"></script>
 </body>
</html>
[padding]
# apps/web/devtools.js
function pretty(obj) {
 if (typeof obj === "string") return obj;
 try {
  return JSON.stringify(obj, null, 2);
 } catch {
  return String(obj);
 }
}
[padding]
async function fetchText(path) {
 const res = await fetch(path);
 const ct = res.headers.get("content-type") || "";
```

```
  if (!res.ok) throw new Error(`${res.status} ${res.statusText}`);
  if (ct.includes("application/json")) return pretty(await res.json());
  return await res.text();
}
[padding]
function setOut(text) {
  const out = document.getElementById("out");
  if (!out) return;
  out.textContent = text;
}
[padding]
async function bind() {
  const map = {
    "btn-health": "/api/health",
    "btn-report": "/api/report",
    "btn-manifest": "/api/web/manifest",
  };
  for (const [id, path] of Object.entries(map)) {
    document.getElementById(id)?.addEventListener("click", async () => {
      setOut(`请求中：${path}`);
      try {
        const text = await fetchText(path);
        setOut(text);
      } catch (e) {
        setOut(`请求失败：${path}\n${String(e)}`);
      }
    });
  }
}
[padding]
window.addEventListener("DOMContentLoaded", bind);
[padding]
# apps/web/index.html
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>园林景观三维可视化设计平台</title>
    <link rel="stylesheet" href="./app.css" />
  </head>
  <body>
    <div id="app"></div>
    <script src="./app.js"></script>
  </body>
</html>
[padding]
# apps/web/student.html
<!doctype html>
<html lang="zh-CN">
```

```
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>评审浏览 - 园林景观三维可视化设计平台</title>
    <meta http-equiv="refresh" content="0; url=./index.html#review" />
  </head>
  <body>
    <p>正在跳转到评审浏览…</p>
  </body>
</html>
# apps/web/teacher.html
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>设计师工作台 - 园林景观三维可视化设计平台</title>
    <meta http-equiv="refresh" content="0; url=./index.html#plan" />
  </head>
  <body>
    <p>正在跳转到设计师工作台…</p>
  </body>
</html>
# scripts/ai_slop_check.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
import runpy
import sys
from pathlib import Path
[padding]
[padding]
def main() -> None:
    workspace_root = Path(__file__).resolve().parents[2]
    script = workspace_root / "scripts" / "ai_slop_check.py"
    if not script.exists():
        raise FileNotFoundError(f"Missing workspace script: {script}")
    sys.path.insert(0, str(workspace_root / "scripts"))
    runpy.run_path(str(script), run_name="__main__")
[padding]
[padding]
if __name__ == "__main__":
    main()
[padding]
# scripts/api_self_check.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
"""
API self-check (local).
```

```
[padding]
This script is intentionally simple and dependency-free. It can be used to:
- verify the built-in server starts;
- verify key endpoints return expected shapes;
- run a short smoke cycle before screenshot capture.
[padding]
Note: The gated pipeline uses `scripts/run_tests.py` and
`scripts/build_materials.py`.
This script is optional but useful during maintenance.
"""
[padding]
import json
import socket
import time
import urllib.error
import urllib.request
from contextlib import closing
from dataclasses import dataclass
from pathlib import Path
from subprocess import Popen
import sys
import os
[padding]
[padding]
def _find_free_port(start: int = 8010) -> int:
    for port in range(start, start + 50):
        with closing(socket.socket(socket.AF_INET, socket.SOCK_STREAM)) as
s:
            try:
                s.bind(("127.0.0.1", port))
                return port
            except OSError:
                continue
    raise RuntimeError("no free port found")
[padding]
[padding]
def _get(url: str, timeout: float = 3.0) -> tuple[int, str, str]:
    req = urllib.request.Request(url, method="GET")
    with urllib.request.urlopen(req, timeout=timeout) as resp:
        data = resp.read()
        ct = resp.headers.get("content-type", "")
        return resp.status, ct, data.decode("utf-8", errors="replace")
[padding]
[padding]
def _post_json(url: str, payload: dict, timeout: float = 3.0) -> tuple[int,
str, str]:
    body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
    req = urllib.request.Request(url, data=body, method="POST")
    req.add_header("Content-Type", "application/json; charset=utf-8")
    with urllib.request.urlopen(req, timeout=timeout) as resp:
```

```
        data = resp.read()
        ct = resp.headers.get("content-type", "")
        return resp.status, ct, data.decode("utf-8", errors="replace")
[padding]
[padding]
def _wait_ready(base_url: str, *, seconds: float = 10.0) -> None:
    deadline = time.time() + seconds
    last_err = None
    while time.time() < deadline:
        try:
            status, _, _ = _get(base_url + "/api/health", timeout=1.5)
            if status == 200:
                return
        except Exception as exc:
            last_err = exc
        time.sleep(0.3)
    raise RuntimeError(f"server not ready: {last_err}")
[padding]
[padding]
@dataclass(frozen=True)
class Check:
    name: str
    ok: bool
    detail: str
[padding]
[padding]
def _assert_json_ok(text: str) -> dict:
    obj = json.loads(text)
    if not isinstance(obj, dict) or obj.get("ok") is not True:
        raise AssertionError("response not ok")
    return obj
[padding]
[padding]
def run_checks(base_url: str) -> list[Check]:
    results: list[Check] = []
[padding]
    def do(name: str, fn) -> None:
        try:
            detail = fn()
            results.append(Check(name=name, ok=True, detail=detail))
        except Exception as exc:
            results.append(Check(name=name, ok=False, detail=str(exc)))
[padding]
    do("health", lambda: str(_get(base_url + "/api/health")[0]))
    do("plants", lambda: str(len(_assert_json_ok(_get(base_url +
"/api/plants")[2]).get("items", []))))
    do("materials", lambda: str(len(_assert_json_ok(_get(base_url +
"/api/materials")[2]).get("items", []))))
    do("issues", lambda: str(len(_assert_json_ok(_get(base_url +
"/api/issues")[2]).get("items", []))))
```

```
    do("report", lambda: "ok" if "关键指标" in _get(base_url + "/api/report")
[2] else "missing")
    do("manifest", lambda: "ok" if "manifest" in _get(base_url +
"/api/web/manifest")[2] else "missing")
[padding]
    def login_and_export() -> str:
        _, _, t = _post_json(base_url + "/api/auth/login", {"username":
"admin", "password": "admin123"})
        obj = _assert_json_ok(t)
        token = obj.get("token", "")
        if not token:
            raise AssertionError("missing token")
        status, _, csv_text = _get(base_url + "/api/export?kind=plants")
        if status != 200 or "编号" not in csv_text:
            raise AssertionError("export not ok")
        return "ok"
[padding]
    do("login+export", login_and_export)
    return results
[padding]
[padding]
def main() -> int:
    project_root = Path(__file__).resolve().parents[1]
    port = _find_free_port()
    env = os.environ.copy()
    env["APP_PORT"] = str(port)
    server = Popen([sys.executable, "apps/api/main.py"], cwd=project_root,
env=env)
    base_url = f"http://127.0.0.1:{port}"
    try:
        _wait_ready(base_url)
        results = run_checks(base_url)
    finally:
        server.terminate()
        try:
            server.wait(timeout=5)
        except Exception:
            server.kill()
[padding]
    failed = [r for r in results if not r.ok]
    for r in results:
        flag = "PASS" if r.ok else "FAIL"
        print(f"[{flag}] {r.name}: {r.detail}")
    return 1 if failed else 0
[padding]
[padding]
if __name__ == "__main__":
    raise SystemExit(main())
[padding]
# scripts/build_materials.py
```

```
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
import runpy
import sys
from pathlib import Path
[padding]
[padding]
def main() -> None:
    workspace_root = Path(__file__).resolve().parents[2]
    script = workspace_root / "scripts" / "build_materials.py"
    if not script.exists():
        raise FileNotFoundError(f"Missing workspace script: {script}")
    sys.path.insert(0, str(workspace_root / "scripts"))
    runpy.run_path(str(script), run_name="__main__")
[padding]
[padding]
if __name__ == "__main__":
    main()
[padding]
# scripts/count_lines.py
"""Count non-empty source-code lines for this project."""
[padding]
from __future__ import annotations
[padding]
from pathlib import Path
[padding]
[padding]
def _resolve_repo_root() -> Path:
    cwd = Path.cwd().resolve()
    if (cwd / "apps").exists() or (cwd / "packages").exists():
        return cwd
    return Path(__file__).resolve().parents[1]
[padding]
[padding]
REPO_ROOT = _resolve_repo_root()
SOURCE_ROOTS = [REPO_ROOT / "apps", REPO_ROOT / "packages", REPO_ROOT /
"scripts"]
TARGET_SUFFIXES = {".py", ".js", ".mjs", ".css", ".html", ".sql", ".json"}
EXCLUDED_DIRS = {
    ".git",
    ".tools",
    "__pycache__",
    "node_modules",
    "run_reports",
    "tmp",
    "copyright-application-materials",
}
[padding]
[padding]
```

```
def iter_source_files() -> list[Path]:
    files: list[Path] = []
    for root in SOURCE_ROOTS:
        if not root.exists():
            continue
        for path in root.rglob("*"):
            if path.is_dir():
                continue
            if any(part in EXCLUDED_DIRS for part in path.parts):
                continue
            if path.suffix.lower() not in TARGET_SUFFIXES:
                continue
            files.append(path)
    return sorted(files)
[padding]
[padding]
def count_lines() -> int:
    total = 0
    for path in iter_source_files():
        try:
            content = path.read_text(encoding="utf-8")
        except UnicodeDecodeError:
            continue
        total += sum(1 for line in content.splitlines() if line.strip())
    return total
[padding]
[padding]
if __name__ == "__main__":
    print(count_lines())
[padding]
# scripts/data_tools.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
"""
Data tools for local JSON state.
[padding]
This module provides tiny maintenance helpers that are safe to run offline:
- validate required keys exist;
- normalize list ordering for stable diffs;
- rebuild minimal seed state if file is missing.
"""
[padding]
import json
from dataclasses import dataclass
from pathlib import Path
from typing import Any
[padding]
[padding]
@dataclass(frozen=True)
```

```
class Result:
    ok: bool
    message: str
[padding]
[padding]
REQUIRED_TOP_KEYS = ["seeded", "sites", "plans", "versions", "plants",
"materials", "lights", "issues", "pavings"]
[padding]
[padding]
def read_state(path: Path) -> dict[str, Any]:
    if not path.exists():
        return {}
    return json.loads(path.read_text(encoding="utf-8"))
[padding]
[padding]
def write_state(path: Path, state: dict[str, Any]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(state, ensure_ascii=False, indent=2),
encoding="utf-8")
[padding]
[padding]
def validate_state(state: dict[str, Any]) -> Result:
    if not isinstance(state, dict):
        return Result(False, "state must be an object")
    missing = [k for k in REQUIRED_TOP_KEYS if k not in state]
    if missing:
        return Result(False, "missing keys: " + ", ".join(missing))
    for k in REQUIRED_TOP_KEYS[1:]:
        if not isinstance(state.get(k), list):
            return Result(False, f"{k} must be a list")
    return Result(True, "ok")
[padding]
[padding]
def sort_state(state: dict[str, Any]) -> dict[str, Any]:
    out = dict(state)
    for key in ("sites", "plans", "versions", "plants", "materials",
"lights", "issues", "pavings"):
        items = out.get(key)
        if not isinstance(items, list):
            continue
        out[key] = sorted(items, key=lambda it: str((it or {}).get("id", "")))
)
    return out
[padding]
[padding]
def main() -> int:
    project_root = Path(__file__).resolve().parents[1]
    state_path = project_root / "apps" / "api" / "data" /
"project_state.json"
    state = read_state(state_path)
```

```
    if not state:
        print("state file missing or empty; nothing to normalize")
        return 0
    res = validate_state(state)
    if not res.ok:
        print("FAIL:", res.message)
        return 1
    sorted_state = sort_state(state)
    write_state(state_path, sorted_state)
    print("PASS: normalized", state_path)
    return 0
[padding]
[padding]
if __name__ == "__main__":
    raise SystemExit(main())
[padding]
# scripts/util_text.py
# -*- coding: utf-8 -*-
from __future__ import annotations
[padding]
[padding]
def wrap_lines(text: str, width: int = 80) -> list[str]:
    out: list[str] = []
    for raw in (text or "").splitlines():
        line = raw.rstrip("\n")
        while len(line) > width:
            cut = line.rfind(" ", 0, width + 1)
            if cut <= 0:
                cut = width
            out.append(line[:cut].rstrip())
            line = line[cut:].lstrip()
        out.append(line)
    return out
[padding]
[padding]
def normalize_ws(text: str) -> str:
    return " ".join((text or "").split())
[padding]
[padding]
def clamp(n: int, *, lo: int, hi: int) -> int:
    return max(lo, min(hi, int(n)))
[padding]
[padding]
def is_blank(text: str | None) -> bool:
    return not (text or "").strip()
[padding]
[padding]
def title_case(text: str) -> str:
    return " ".join(w[:1].upper() + w[1:] for w in normalize_ws(text)
.split(" ") if w)
```